Dossier d'architecture technique

MesurePlan — conception, modules, données, algorithmes, qualité et déploiement

Version 1.3.0 — 25 octobre 2026

Application MesurePlan — métré, annotation et impression à l'échelle de plans PDF vectoriels.
Licence AGPL-3.0 (imposée par PyMuPDF).

Table des matières

1 Objet et périmètre 3
1.1 Objet du document 3
1.2 Besoin 3
1.3 Hors périmètre 3
1.4 Exigences non fonctionnelles 3
2 Vue d'ensemble 4
2.1 Couches 4
2.2 Dépendances externes 4
3 Modules 5
4 Conventions géométriques 6
4.1 Repère 6
4.2 Échelle 6
4.3 Rotation 6
4.4 Grandeurs « papier » 6
5 Modèle de données 7
5.1 Session (`plan.pdf.mesures.json`) 7
5.2 Configuration (`config.json`) 7
5.3 Autres persistances 7
6 Algorithmes 8
6.1 Extraction des traits et accrochage 8
6.2 Détection de l'échelle du cartouche 8
6.3 Motifs de remplissage 8
6.4 Édition 8
6.5 Impression à l'échelle et mosaïque 8
6.6 Historique 8
7 Flux principaux 9
8 Qualité 10
8.1 Tests automatisés 10
8.2 Validation complémentaire réalisée 10
8.3 Recette manuelle recommandée 10
9 Déploiement 11
9.1 Installation développeur 11
9.2 Exécutables autonomes 11
10 Risques, limites et évolutions 12
10.1 Limites connues 12
10.2 Registre des risques 12
10.3 Décisions d'architecture 12
10.4 Pistes d'évolution 12

1 Objet et périmètre

1.1 Objet du document

Ce dossier décrit l'architecture de MesurePlan : ses modules et leurs dépendances, le modèle et les formats de données, les conventions géométriques, les algorithmes principaux, la stratégie de test, le déploiement, ainsi que les limites et risques connus. Il s'adresse à toute personne (ou IA) chargée de maintenir ou de faire évoluer l'application.

1.2 Besoin

Exploiter des plans d'architecte reçus au format **PDF vectoriel**, sans accès aux fichiers natifs (DWG, DXF, IFC) :

- mesurer distances, chaînes de cotes, surfaces, périmètres et angles **à l'échelle** ;
- corriger l'échelle du cartouche par calibration sur une ou plusieurs cotes connues ;
- annoter le plan (flèches, lignes, formes, textes, motifs techniques de remplissage) ;
- éditer après coup : déplacement, sommets, rotation ;
- produire un PDF annoté et imprimer à une échelle choisie, sur tout format de papier ;
- fonctionner sous Linux, Windows et macOS, y compris sans Python installé.

1.3 Hors périmètre

Calcul de volumes, modélisation 3D, plans scannés (pris en charge sans accrochage), écriture dans le PDF d'origine (jamais modifié), travail collaboratif.

1.4 Exigences non fonctionnelles

Exigence	Réponse architecturale
Exactitude des mesures	Géométrie conservée en coordonnées PDF natives (pt) ; échelle appliquée au calcul ; accrochage exact aux traits vectoriels.
Traçabilité de l'échelle	Origine affichée (cartouche lu, saisi, calibré), écart au cartouche, dispersion entre références.
Ergonomie	Valeur en direct près du curseur, accrochage CAO, poignées d'édition, raccourcis à une touche, annuler / rétablir.
Fidélité écran / papier	Primitives calculées une seule fois par le modèle pur, consommées à l'identique par Qt et par PyMuPDF.
Robustesse	Écriture atomique de la session, sauvegarde automatique, journal tournant, exceptions non gérées journalisées.
Testabilité	Noyau sans dépendance graphique : 39 tests unitaires exécutables partout.
Portabilité	Python + Qt (PySide6) ; exécutables PyInstaller construits par intégration continue sur les trois systèmes.

2 Vue d'ensemble

2.1 Couches

L'application suit une architecture en trois couches : un **noyau pur** (aucun import Qt ni PyMuPDF au chargement), une couche d'**accès PDF** et une couche de **présentation** Qt. Le noyau porte toute la logique métier, ce qui le rend testable sans affichage.

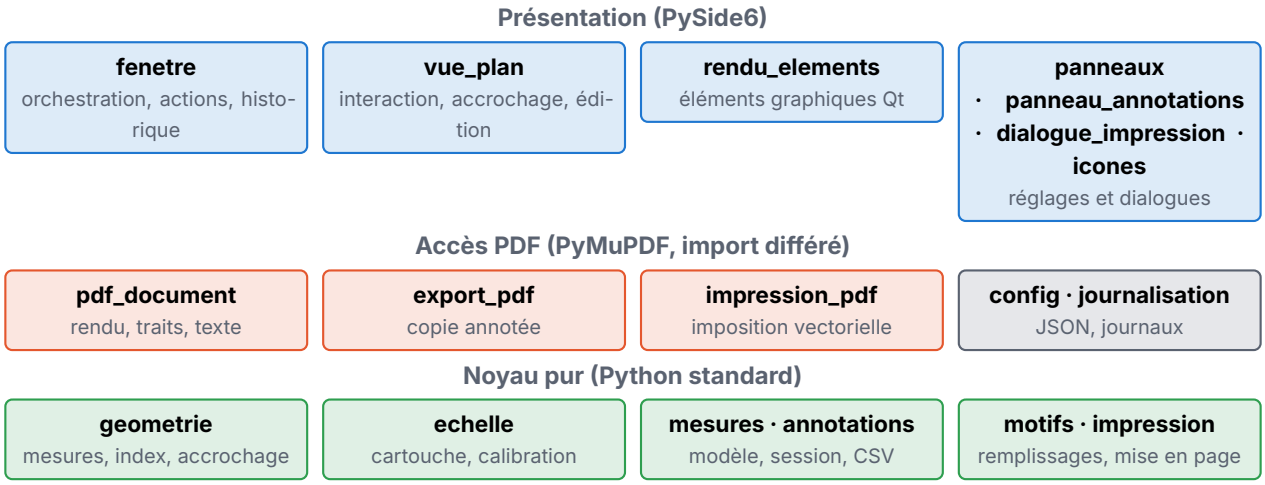


Fig. 1. – Couches et modules. Une couche n'importe que des couches de niveau inférieur.

2.2 Dépendances externes

Composant	Version	Licence	Usage
Python	≥ 3.10	PSF	Langage (types `X | Y`, `match` non utilisé).
PySide6	≥ 6.6	LGPL v3	Interface, impression directe (QtPrintSupport).
PyMuPDF	≥ 1.24	AGPL v3	Rendu, extraction des traits (`get_drawings`), texte, écriture PDF.
PyInstaller	≥ 6.6	GPL + exception	Construction des exécutables (non embarqué à l'exécution).
pytest	≥ 8	MIT	Tests (développement uniquement).

> **Attention**
> La licence AGPL de PyMuPDF s'étend à l'application distribuée : diffusion sous licence compatible AGPL, code source fourni. Une licence commerciale PyMuPDF (Artifex) lève cette contrainte.

3 Modules

Module	Responsabilité	Testé
`geometrie`	Distances, aire (lacet), centroïde, angles, projections, intersections, rotation, contrainte d'angle, index spatial à grille, accrochage.	oui
`echelle`	Modèle `EchellePage` (dénominateur du cartouche, références de calibration), facteur m/pt, écart, dispersion ; détection de l'échelle dans le texte.	oui
`mesures`	Modèle `Mesure`, `Session` (mesures, annotations, échelles par page), sérialisation JSON v2, export CSV, mise en forme française.	oui
`annotations`	Modèle `Annotation` et `Style` : contour, rotation, poignées, distance (sélection), pointes de flèches, texte compatible PDF, conversion des mesures en primitives.	oui
`motifs`	24 motifs techniques générés dans un repère local puis découpés exactement par le contour.	oui
`impression`	Facteur d'agrandissement, orientation automatique, mosaïque avec recouvrement, barre d'échelle.	oui
`pdf_document`	Ouverture, normalisation des pages tournées, rendu raster, traits vectoriels → segments annotés, texte → détection d'échelle.	partiel[1]
`export_pdf`	Copie du PDF avec éléments incrustés ou annotations PDF natives (repli automatique en incrustation).	simulé[2]
`impression_pdf`	PDF d'impression : `show_pdf_page` par feuille, repères, légende, barre d'échelle, segment de contrôle, plan d'assemblage.	simulé[2]
`vue_plan`	Zoom/panoramique, rendu de détail différé, outils de tracé, accrochage, sélection, édition (déplacement, sommets, rotation).	non[3]
`rendu_elements`	Éléments `QGraphicsItem` : étiquettes, marqueurs, poignées, annotations (rotation, motifs mis en cache), mesures.	non[3]
`fenetre`	Fenêtre principale : actions, panneaux, session, historique annuler/rétablir, export, impression.	non[3]
`config`, `journalisation`, `app`	Configuration JSON fusionnée aux valeurs par défaut, journal console + fichier tournant, point d'entrée.	non

[1] La conversion `get_drawings` → segments est testée sur données simulées. [2] Chemins de code exercés avec un PyMuPDF simulé ; non exécutés contre la bibliothèque réelle dans l'environnement de développement. [3] Couche graphique : vérifiée par analyse statique, à valider au premier lancement (voir § Qualité).

4 Conventions géométriques

4.1 Repère

Toutes les géométries sont stockées dans le **repère de la page PDF** : unité le point typographique (1 pt = 1/72 pouce = 0,3528 mm), origine au coin supérieur gauche, axe Y **vers le bas**. Les pages tournées (`/Rotate`) sont normalisées en mémoire à l'ouverture (`Page.remove_rotation()`), ce qui donne un repère unique pour le rendu, les traits, le texte et l'export.

4.2 Échelle

Pour une page au cartouche 1:N : $f_{\text{cartouche}} = N \times 0{,}\frac{0254}{72}$ mètres par point. Avec des références de calibration $\left(L_i^{\text{pt}}, L_i^{\text{réel}}\right)$:

$$f = \frac{1}{n}\sum_i \frac{L_i^{\text{réel}}}{L_i^{\text{pt}}}, \quad \text{écart} = \frac{f}{f_{\text{cartouche}}} - 1, \quad \text{dispersion} = \frac{\max_i f_i - \min_i f_i}{f}.$$

Longueur réelle = longueur en pt × f ; surface réelle = surface en pt² × f². L'échelle étant supposée **isotrope**, les angles ne dépendent pas de f.

4.3 Rotation

Angles en degrés, **positifs dans le sens trigonométrique à l'écran** (donc horaire dans le repère mathématique Y vers le bas) :

$$x' = c_x + d_x\cos\theta + d_y\sin\theta, \quad y' = c_y - d_x\sin\theta + d_y\cos\theta.$$

Rectangle et ovale stockent deux coins **avant rotation** et un angle (rotation autour du centre) ; lignes et flèches sont tournées en déplaçant leurs extrémités. Qt tournant dans le sens horaire à l'écran, la vue applique `QTransform.rotate(-θ)`.

4.4 Grandeurs « papier »

Épaisseurs de trait, tailles de police et espacements de motifs sont exprimés en **points papier** : ils grandissent avec le zoom comme sur le plan, et sont identiques dans le PDF exporté.

5 Modèle de données

5.1 Session (`plan.pdf.mesures.json`)

Fichier JSON écrit à côté du PDF, de façon atomique (fichier temporaire puis remplacement), après chaque modification si `session.sauvegarde_auto` est vrai. Les formats 1 (sans annotations) et 2 sont lisibles.

```
{
  "format": 2,
  "application": "MesurePlan",
  "enregistre_le": "2026-10-25T13:40:00",
  "chemin_pdf": "/home/bernard/plans/RDC.pdf",
  "denominateur_defaut": 100,
  "prochain_identifiant": 12,
  "echelles": {
    "0": {
      "denominateur_cartouche": 100,
      "origine_cartouche": "lue sur le plan",
      "references": [
        {"p1": [120.0, 150.0], "p2": [403.5, 150.0], "longueur_reelle_m": 10.0}
      ]
    }
  },
  "mesures": [
    {"identifiant": 3, "type": "surface", "page": 0,
     "nom": "Séjour", "nom_personnalise": true,
     "points": [[125.7, 155.7], [290.1, 155.7], [290.1, 371.2], [125.7, 371.2]]}
  ],
  "annotations": [
    {"identifiant": 7, "type": "rectangle", "page": 0, "angle": 30.0,
     "points": [[200, 300], [320, 360]], "texte": "", "centre": false,
     "style": {"couleur_trait": "#D81B60", "couleur_fond": null,
               "opacite_fond": 0.35, "epaisseur": 1.5, "trait": "plein",
               "taille_police": 10.0, "cadre_texte": true, "double_fleche": false,
               "motif": "beton", "motif_angle": 0.0, "motif_echelle": 1.0,
               "motif_couleur": null, "motif_epaisseur": 0.3}}
  ]
}
```

Le compteur `prochain_identifiant` est commun aux mesures et aux annotations : un identifiant désigne un élément unique, ce qui simplifie la sélection mixte. Les clés de style inconnues sont ignorées (compatibilité ascendante).

5.2 Configuration (`config.json`)

Recherche dans l'ordre : variable `MESUREPLAN_CONFIG`, `./config/config.json`, `~/.mesureplan/config/config.json` (créé au premier lancement). Le fichier utilisateur est **fusionné** récursivement avec la configuration embarquée : seules les clés modifiées sont nécessaires. Sections : `journalisation`, `rendu`, `accrochage`, `contrainte`, `echelle`, `affichage`, `session`, `export`.

5.3 Autres persistances

- **QSettings** (registre Windows, `~/.config` Linux, `plist` macOS) : géométrie de la fenêtre, état des panneaux, dernier dossier, dernier style d'annotation ;
- **journal** : `logs/mesureplan.log` relatif au dossier de base de la configuration, fichier tournant ; chaque ligne porte horodatage, niveau, utilisateur système (`getpass.getuser()`), module et message.

6 Algorithmes

6.1 Extraction des traits et accrochage

`Page.get_drawings()` fournit lignes, rectangles, quadrilatères et courbes de Bézier. Chaque élément devient une liste de segments annotés de drapeaux : vraie extrémité A ou B, segment droit (milieu significatif). Les courbes sont discrétisées en 8 segments, dont seules les extrémités réelles sont accrochables. Les segments sont indexés dans une **grille régulière** (pas = plus grande dimension / 300), chaque segment étant échantillonné tous les demi-pas. Pour une position souris, on interroge les cellules couvrant le rayon de tolérance (12 px écran par défaut), puis :

1. candidats ponctuels : extrémités, milieux, intersections des 40 segments les plus proches, sommets des éléments existants ;
2. le plus proche l'emporte ; à défaut, projection sur le trait le plus proche ; sinon point libre.

6.2 Détection de l'échelle du cartouche

Expression `1 [/:] N` non collée à d'autres chiffres (exclut les dates `12/03/2026` et `Lot 1/3`). Score 3 si l'occurrence est précédée d'un mot-clé (« Éch. », « Échelle », « Scale ») dans les 20 caractères précédents ; score 1 si N est une échelle usuelle ; ignorée sinon. L'échelle de plus fort score est proposée **et signalée comme non vérifiée** : un PDF exporté « ajusté à la page » garde sa mention 1/100 tout en étant faux.

6.3 Motifs de remplissage

Chaque motif est un générateur `(R, s) → primitives` dans un repère local couvrant le carré $[-R, R]^2$ (R = demi-diagonale de la boîte du contour, s = pas de base × échelle choisie). Les primitives sont tournées (angle propre du motif + angle choisi) autour du centre de la forme, puis **découpées exactement** par le contour : pour chaque segment, intersections avec les arêtes, tri des paramètres, conservation des intervalles dont le milieu est intérieur (règle pair-impair). La tolérance sur le paramètre d'arête traite le cas d'un trait passant exactement par un sommet du contour. Les motifs irréguliers (béton, gravier…) utilisent une graine pseudo-aléatoire fixe **par cellule** : dessin stable et reproductible. Le volume est borné à environ 25 000 primitives (espacement augmenté sinon, signalé au journal) ; un cache LRU de 300 entrées évite de recalculer à chaque rafraîchissement de la vue.

6.4 Édition

Le glisser travaille sur une **copie** de la géométrie initiale : à chaque mouvement, la nouvelle géométrie est recalculée depuis l'état initial (pas d'accumulation d'erreurs), l'élément est redessiné seul avec sa valeur en direct, puis un unique signal `geometrie_modifiee` est émis au relâchement. Redimensionner un rectangle tourné fixe le coin opposé : nouveau centre = milieu des deux coins, coordonnées « avant rotation » obtenues par rotation inverse autour de ce centre.

6.5 Impression à l'échelle et mosaïque

Agrandissement $k = \frac{f}{N_{\text{impression}} \times 0{,}3528\ \text{mm}}$. Pour chaque orientation candidate, zone utile = feuille − marges − pied de page ; nombre de feuilles par axe $n = \left\lceil \frac{L - r}{u - r} \right\rceil$ (recouvrement r, zone utile u) ; l'orientation retenue minimise le nombre de feuilles, puis le papier perdu. Chaque feuille reçoit sa portion via `show_pdf_page(clip)`, qui **conserve le dessin vectoriel**.

6.6 Historique

Annuler / rétablir par instantanés : l'état complet de la session (dictionnaire JSON) est empilé avant chaque modification, 100 niveaux au plus. Simple et sûr ; le coût mémoire reste faible (quelques dizaines de kilo-octets par état).

7 Flux principaux

Flux	Déroulement
Ouverture	`DocumentPDF` (normalisation des rotations) → chargement de `plan.pdf.mesures.json` s'il existe → première page : rendu du fond, indexation des traits, lecture de l'échelle.
Mesure	Outil → clics accrochés → `mesure_terminee` → `Session.ajouter_mesure` → instantané d'historique → sauvegarde → rafraîchissement (vue, tableau, totaux).
Calibration	Trait de référence → saisie de la cote → `Reference` ajoutée → facteur recalculé → toutes les mesures mises à jour (aucun retracé).
Édition	Poignée ou corps de l'élément → copie → recalcul en direct → `geometrie_modifiee` → modèle mis à jour → sauvegarde.
Export PDF	Copie en mémoire → mesures converties en primitives → incrustation ou annotations PDF (repli) → enregistrement sous un autre nom.
Impression	Document annoté figé (octets) → mise en page → feuilles vectorielles + repères → PDF ; impression directe éventuelle par rendu 300 dpi via `QPrinter`.

8 Qualité

8.1 Tests automatisés

39 tests (pytest) couvrent le noyau : géométrie et accrochage (16), annotations et persistance (8), mise en page d'impression (5), rotation, poignées et motifs (10), dont un test vérifiant que les 24 motifs restent intérieurs à un contour circulaire. Ils ne nécessitent ni affichage ni PyMuPDF.

8.2 Validation complémentaire réalisée

- chaîne complète échelle → accrochage → surface → calibration validée sur le plan d'essai `exemples/plan_essai.pdf` (traits extraits par pdfplumber à la place de PyMuPDF) : 44,08 m², 10,00 m, 150,00° ; écart de +3,09 % détecté sur la page imprimée à 97 % ;
- export PDF et impression exercés avec un PyMuPDF simulé (tous les types, deux modes) ;
- analyse statique (noms non définis, imports inutilisés) de tous les modules.

> **Attention**
> L'environnement de développement n'avait pas accès à PyPI : **la couche graphique et les appels réels à PyMuPDF n'y ont jamais été exécutés.** Premier lancement à surveiller (journal). L'intégration continue exécute les tests sur les trois systèmes mais ne lance pas l'interface.

8.3 Recette manuelle recommandée

Ouvrir le plan d'essai ; vérifier sur la page 1 la façade (10,00 m), le séjour (44,08 m²) et l'angle du mur biais (150°) ; calibrer la page 3 sur la façade (écart affiché +3,09 %) ; créer chaque annotation, la faire tourner, éditer ses sommets, appliquer un motif ; exporter dans les deux modes et ouvrir dans Okular ; imprimer au 1:50 sur A4 et mesurer le segment de contrôle de 100 mm.

9 Déploiement

9.1 Installation développeur

`pip install -e ".[dev]"` puis `mesureplan` ou `python -m mesureplan`.

9.2 Exécutables autonomes

Recette `packaging/mesureplan.spec` (PyInstaller, mode dossier, modules Qt inutiles exclus, sans UPX pour éviter les faux positifs antivirus). PyInstaller ne compile pas en croisé : le workflow `.github/workflows/executables.yml` construit sur Windows, macOS arm64, macOS x86_64 et Ubuntu 22.04, exécute les tests, archive et publie les archives sur une étiquette `v*`. Scripts locaux : `packaging/construire.sh` (Linux, macOS) et `packaging/construire.ps1` (Windows).

Système	Remarques
Windows	Exécutable non signé : alerte SmartScreen au premier lancement (« Informations complémentaires » → « Exécuter quand même »). Signature Authenticode possible avec un certificat de signature de code.
macOS	Paquet `.app` non signé ni notarisé : clic droit → « Ouvrir » la première fois, ou `xattr -dr com.apple.quarantine MesurePlan.app`. Ouverture de PDF depuis le Finder gérée (`QFileOpenEvent`).
Linux	Construit sur glibc 2.35 : compatible avec les distributions plus récentes (Manjaro inclus). Lanceur `packaging/mesureplan.desktop` à adapter.

10 Risques, limites et évolutions

10.1 Limites connues

- échelle isotrope supposée (un PDF déformé est détecté par la dispersion, pas corrigé) ;
- police PDF Helvetica standard (jeu Latin-1) : caractères hors jeu substitués ;
- mode « annotations PDF » : une seule opacité par annotation ; motifs toujours incrustés sous l'annotation ;
- texte non orientable ; pas de calques ni de groupement d'éléments ;
- impression : précision finale dépendante du pilote (le segment de contrôle permet de la vérifier).

10.2 Registre des risques

Risque	Niveau	Traitement
Couche Qt non exécutée en développement	élevé	Recette manuelle au premier lancement, journal détaillé, `sys.excepthook` journalisé.
Évolution d'API PyMuPDF	moyen	Import différé, version minimale fixée, repli « incrusté » en export.
Licence AGPL	moyen	Documentée ; licence commerciale Artifex si diffusion fermée.
Performance des motifs sur grandes surfaces	faible	Plafond de primitives, cache LRU.
Plans scannés	faible	Message explicite, mesure sans accrochage.

10.3 Décisions d'architecture

Décision	Choix	Raison
Stockage géométrique	Points PDF natifs	L'échelle se corrige sans retracer ; exactitude conservée.
Rendu des motifs	Primitives découpées en Python	Identité stricte écran / PDF ; pas de dépendance au découpage de chaque moteur.
Historique	Instantanés JSON	Simplicité, aucune commande inverse à maintenir.
Export	Copie, jamais d'écriture en place	Préserve le document de l'architecte.
Impression	PDF vectoriel imposé	Netteté, indépendance du pilote ; impression directe en option.
Distribution	PyInstaller + CI multi-OS	Pas de Python requis chez l'utilisateur ; construction reproductible.

10.4 Pistes d'évolution

Import DXF/SVG ; calques et groupes ; orientation du texte ; cotation automatique (lignes de cote architecturales) ; métré par pièce avec tableau exportable en Excel ; motifs paramétrables par fichier ; signature des exécutables ; tests d'interface avec `pytest-qt`.